

Report: Text to Python Code Generation task

1. Project Overview

The objective of this project is to implement and compare three neural sequence to sequence architectures using PyTorch to evaluate their ability to generate Python source code from natural language docstrings. It shows the improvement for better code generation from vanilla RNN model to LSTM and then Attention based LSTM architecture.

2. Dataset

Source	Hugging Face dataset: CodeSearchNet Python
Splits	10000 training, 1000 validating, 1000 testing,
Token limits	Docstring - 50, code - 80

3. Model Architectures

Model 1: Seq2SeqRNN

- **ENCODER:** Standard RNN - 1 layer (Unidirectional)
- **DECODER:** Standard RNN - 1 layer (Unidirectional)
- **ATTENTION:** None; uses the final hidden state as a fixed-length context vector.

Model 2: Seq2SeqLSTM

- **ENCODER:** LSTM - 1 layer (Unidirectional)
- **DECODER:** LSTM - 1 layer (Unidirectional)
- **ATTENTION:** None; uses final hidden as a fixed-length context vector.

Model 3: Seq2SeqLSTMAttention

- **ENCODER:** LSTM - 1 layer (**Bidirectional**)
- **DECODER:** LSTM - 1 layer (Unidirectional)
- **ATTENTION:** **Bahdanau Attention** (Additive Attention); dynamically computes a weighted sum of all encoder outputs for every decoder step.

4. Evaluation Result

Model	BLEU Score	Exact Match (%)	Token Acc (%)
Vanilla RNN	0.18	0.00	9.26
LSTM	0.79	0.00	10.19
LSTM + Attention	2.30	0.00	10.96

The evaluation results tell us that the trend of Bleu score among vanilla RNN, LSTM, LSTM + Attention is in correct order.

BLEU Score Analysis

Vanilla RNN (0.18): It is a low score. It performs poorly.

LSTM (0.79): It shows improvement due to better handling of long-term dependencies. It can maintain context across longer sequences compared to vanilla RNN.

LSTM + Attention (2.30): It outperforms both RNN and plain LSTM. Attention allows the model to focus on relevant parts of the docstring when generating each token, which increases n-gram matches.

Overall, the attention mechanism clearly benefits code generation and bleu score.

Exact Match (EM)

All models score **0.00%**, indicating that none of the generated code samples were fully identical to reference.

Token Accuracy

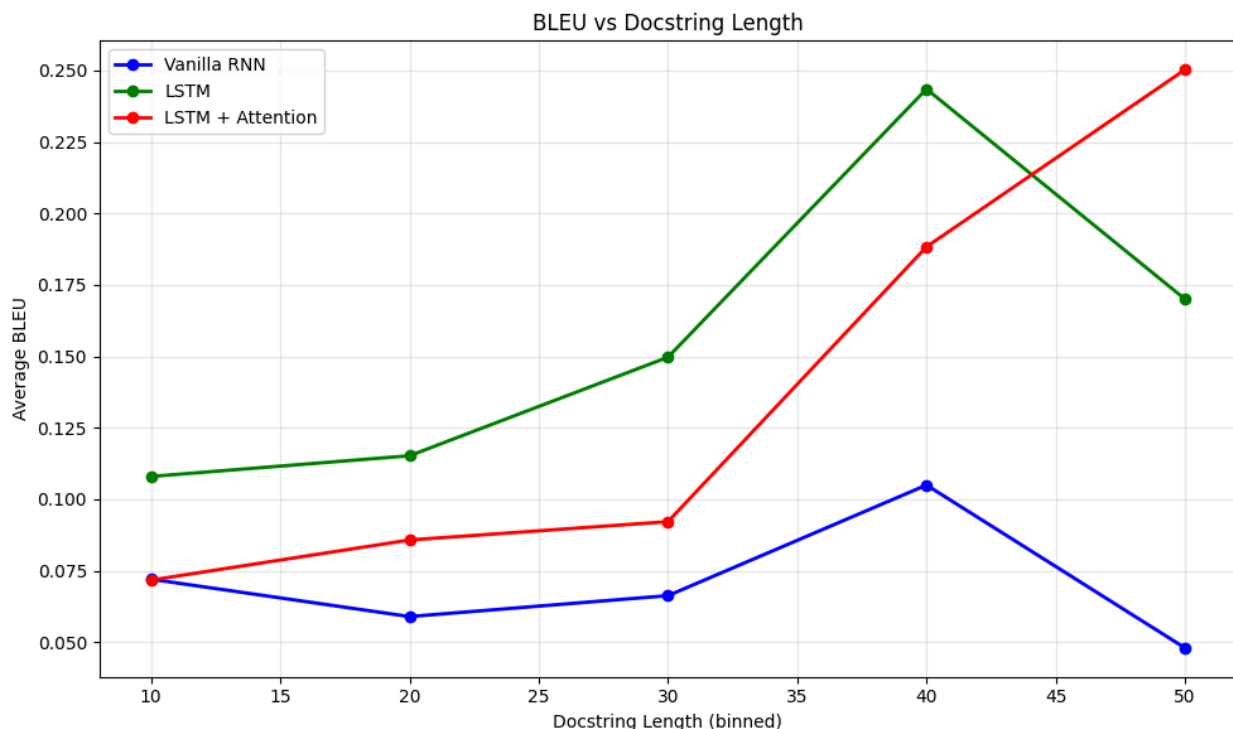
Vanilla RNN: 9.26%. It is very low.

LSTM: 10.19%. It achieves slight improvement due to better sequence modeling.

LSTM + Attention: 10.96%. This time attention gives further improvement, but overall token-level accuracy is still low.

For the BLEU vs Docstring Length:

LSTM with attention achieves higher average bleu score than others for higher docstring length here 50).



Comparative Error Analysis

The comparative error analysis reveals clear differences in generation behavior across the three models.

All models exhibit a high **Operator/Variable Error rate (72%)**, indicating that correctly predicting identifiers and operators remains a major challenge regardless of architecture. This suggests that lexical precision and variable naming are not significantly improved by recurrence depth or attention mechanisms alone.

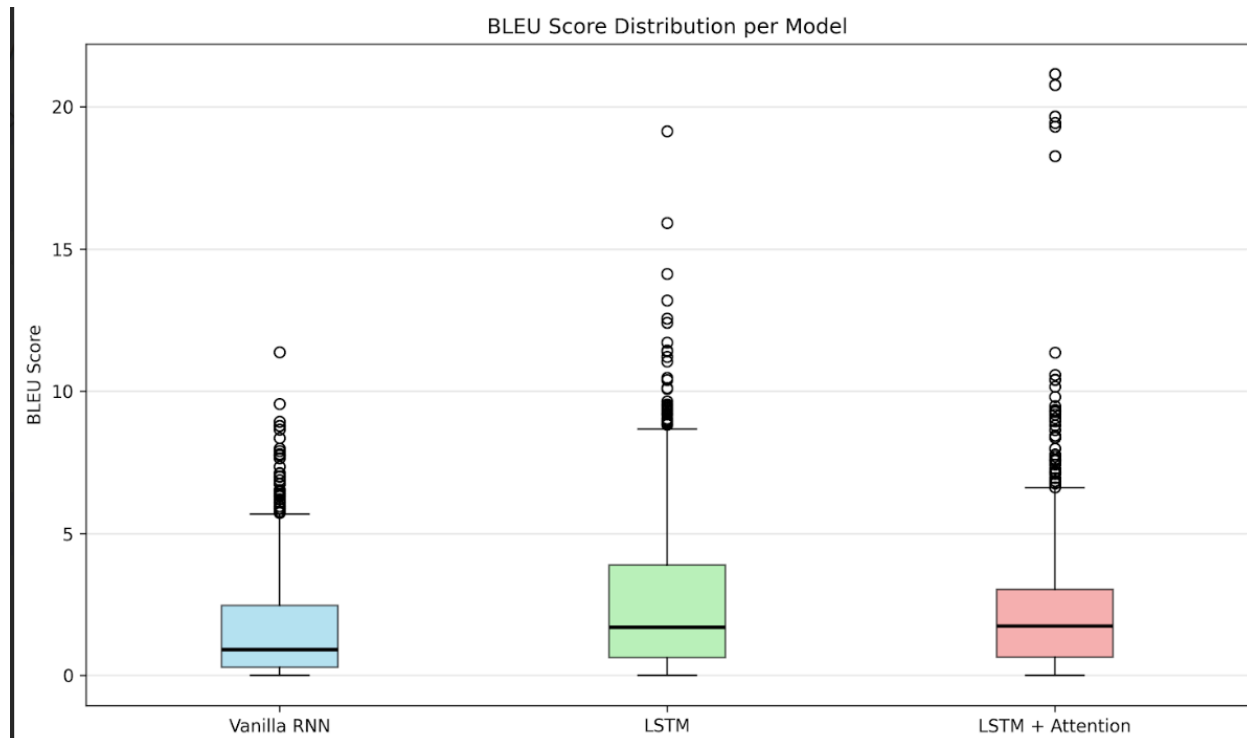
However, differences emerge in **Missing Tokens** and **Extra Tokens**:

- The **Vanilla RNN** shows the worst performance, with **100% missing tokens** and no extra tokens. This indicates that the model frequently produces incomplete outputs, likely due to limited memory capacity and difficulty handling longer dependencies.
- The **LSTM** reduces missing tokens slightly (96%) but introduces a small number of extra tokens (4%). This suggests improved sequence modeling compared to the vanilla RNN, though outputs remain largely incomplete.
- The **LSTM + Attention** model demonstrates the most balanced behavior. Missing tokens drop significantly to **73%**, but extra tokens increase to **27%**. This indicates that attention helps the model generate more complete sequences by better utilizing source

information, though it sometimes over-generates tokens.

Overall, the results suggest that attention improves coverage of the source sequence and reduces under-generation, but may introduce mild over-generation. Despite improvements, high operator/variable error rates across all models indicate that precise token-level accuracy remains a key limitation.

Bleu Score distribution per model



Though bleu score distribution shows that LSTM has better score than LSTM + Attention, bleu score for the train corpus overall achieves higher score in LSTM + Attention than LSTM. The distribution shows that we have outliers in both models. But the LSTM + Attention has an outlier with a high bleu score.

5. Heatmap

For example-1:

The attention heatmap shows that the decoder places higher weights on semantically relevant source tokens during code generation. However, attention remains concentrated on early tokens, indicating limited alignment sharpness.

Observation:

- Source tokens: "Make", "a", "character", "or", "block", "device", "called", "targetpath", "."

- Generated tokens: Python function code tokens like def, get_outputs, self, return, =, etc.

Key Patterns:

1. **Strong attention on first token “Make”**
 - Rows like def, ,, and) have bright attention on “Make”.
 - Indicates the model is relying heavily on the **first word** of the docstring.
2. **Some distributed attention across mid tokens**
 - Rows like return or self have moderate weights on middle tokens like device, called.
 - This is positive: model partially uses context to generate correct assignments and attributes.
3. **Weak attention on later tokens**
 - Tokens like targetpath and . are mostly ignored.
 - This suggests the model is not fully attending to all semantically important words.

Interpretation:

- Attention works partially.
- There’s a bias toward early tokens, but later words sometimes contribute.
- Generated code may be okay for simple structures but may fail for nuanced instructions.

For example-2:

Attention works but is shallow.

Observation:

- Source tokens: "Sort", "label", "names", "putting", "well", "known", "resource", "labels", "first", "."
- Generated tokens: Python function code tokens like def, get_outputs, self, return, etc.

Key Patterns:

1. **Very strong vertical stripe on first token “Sort”**
 - Almost all generated code tokens have peak attention here.
 - This is a classic **attention collapse** — the model ignores most of the docstring.
2. **Very weak attention on semantically meaningful later tokens**
 - Words like label, names, resource are barely attended to.
 - This suggests the model isn’t aligning code with specific docstring semantics.
3. **Overall alignment is poor**

- Generated code is likely generic or based on memorized patterns, not precise instructions.

Interpretation:

- Example 2 shows **collapsed attention** — most of the input is ignored.
- This is worse than Example 1 because semantic mapping from docstring → code is minimal.

For example-3:

There is a **very strong vertical stripe on the first column**. That means that for almost ALL generated tokens, the model is attending heavily to the FIRST source token.

So, attention visualization reveals that the model tends to focus disproportionately on early source tokens, with limited alignment sharpness across the full sequence. In longer examples, attention occasionally collapses to a single dominant source token which indicates limited contextual utilization.

6. Conclusion

The models achieve very low accuracy and bleu score. But the project shows that LSTM achieves better scoring than the vanilla RNN. LSTM with attention achieves better scoring than the LSTM. It shows that considering long range dependencies and attention improves model's accuracy and bleu scoring.